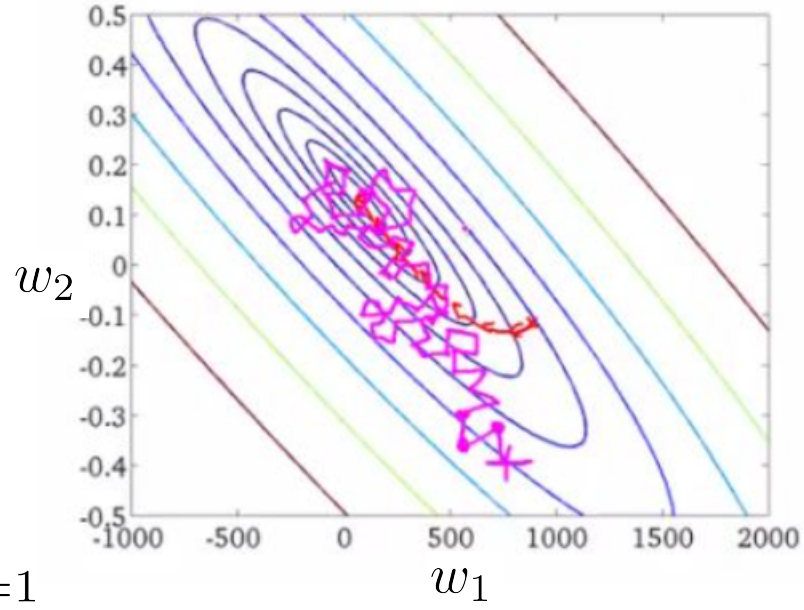
Recap: Stochastic Gradient Descent (SGD)

Stochastic Gradient Descent (SGD) motivated by the application in machine learning.

$$f(\boldsymbol{w}) = \frac{1}{m}\sum_{i=1}^{m} h_i(\boldsymbol{w}; \mathbf{x}^{(i)}, y^{(i)})$$

$$\Rightarrow \nabla f(\boldsymbol{w}) = \frac{1}{m}\sum_{i=1}^{m} \nabla h_i(\boldsymbol{w}; \mathbf{x}^{(i)}, y^{(i)})$$

Need to compute *m* gradients! This is very costly when *m* is large.



Intuition of picking a random function $h_i(\boldsymbol{w})$ among $\{h_i(\boldsymbol{w})\}_{i=1}^{m}$

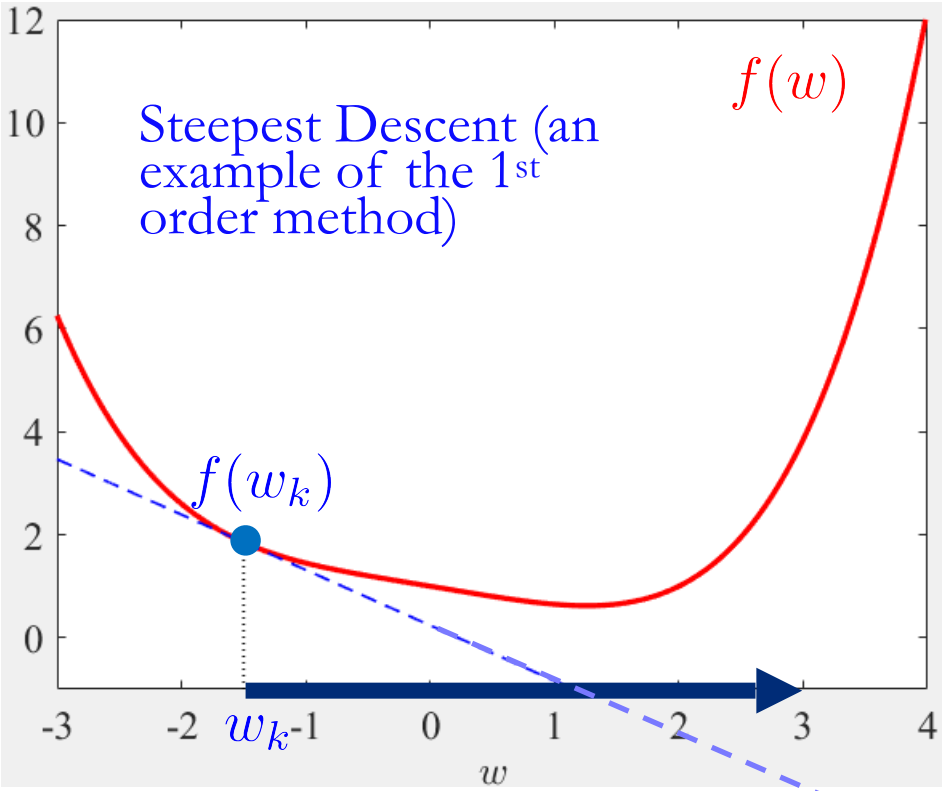
In machine learning, we often assume that all the training data samples are independent random draws from the same joint distribution of (**x**, y). So $h_i(\boldsymbol{w})$ is no special to $h_j(\boldsymbol{w})$, probabilistically.

"Average Direction": $$\mathbb{E}\left[\ \nabla h_i(\boldsymbol{w}; \mathbf{x}^{(i)}, y^{(i)})\ \right] \approx \frac{1}{m}\sum_{i=1}^{m} \nabla h_i(\boldsymbol{w}; \mathbf{x}^{(i)}, y^{(i)}) = \nabla f(\boldsymbol{w})$$

1. Cheaper Computation
2. A Side Benefit
3. Can apply fixed learning rate, e.g., α=0.1

Recap: Steepest Descent vs. Newton's Method---at High Level

$$e.g.,\ \min_w\ f(w) = \frac{1}{20}w^4 - \frac{2}{5}w + 1$$

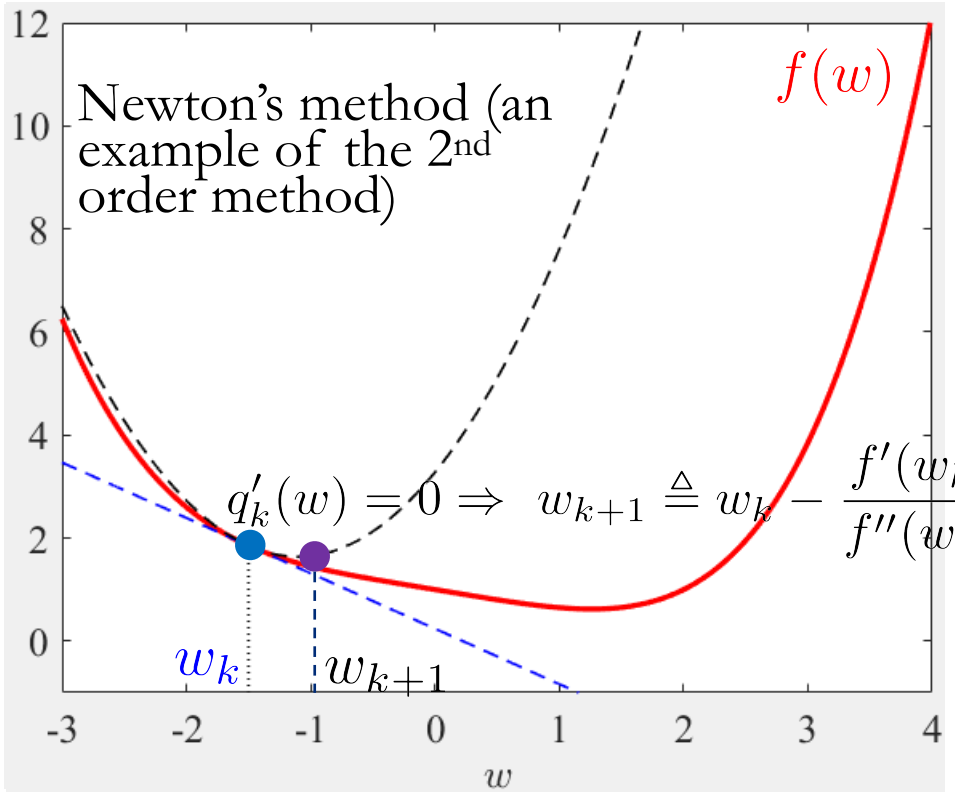


$$f(w) \approx f(w_k) + f'(w_k)(w - w_k) \triangleq l_k(w)$$

Minimizing the (linear) 1st order Taylor approximation of $f(\mathbf{w})$ at $\boldsymbol{w}=\boldsymbol{w}_k$ will lead to $-\infty$. Need to restrict step size $\quad w_{k+1} = w_k - \alpha f'(w_k)$

Build a model (proxy) for $f(w)$ locally at w_k in each iteration, using Taylor expansion. Then instead of min $f(w)$, we "min" the proxy. This leads to the 1st order and 2nd order method.

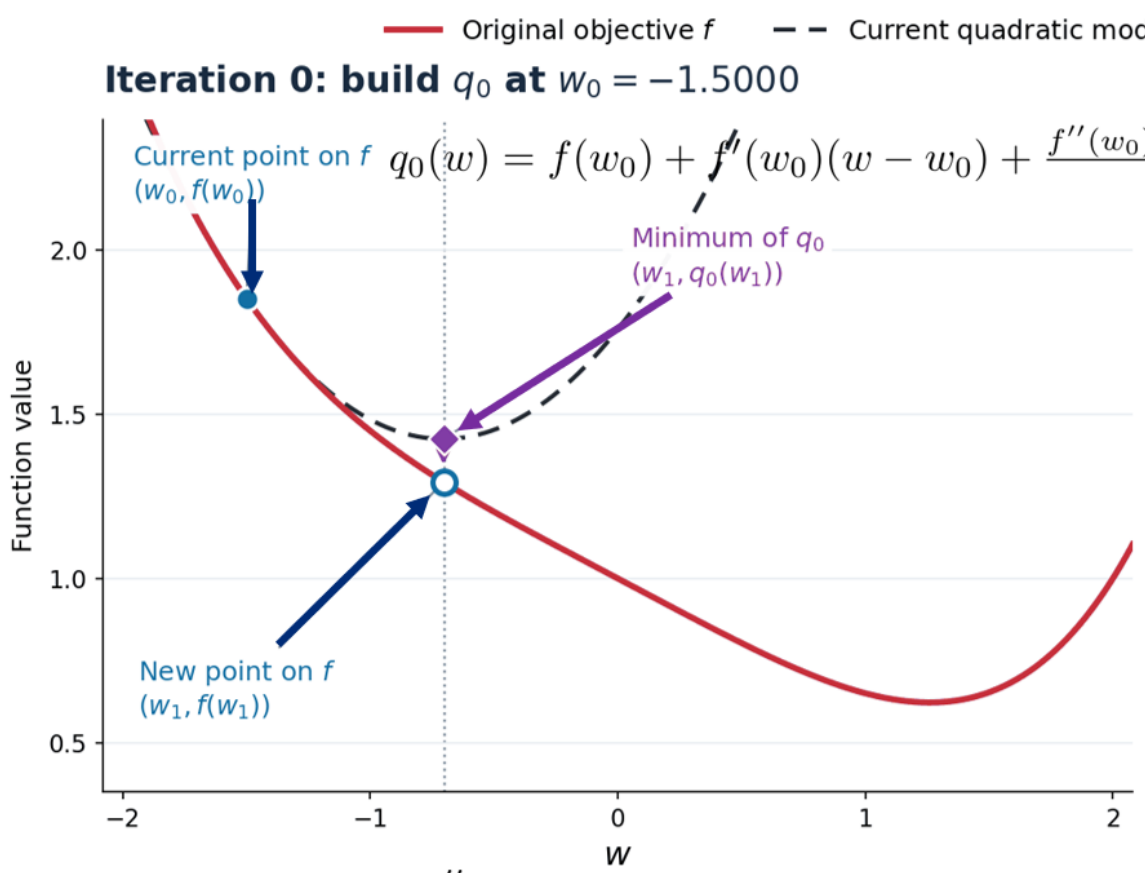
$$f(w) \approx f(w_k) + f'(w_k)(w - w_k) + \frac{f''(w_k)}{2}(w - w_k)^2 \triangleq q_k(w)$$

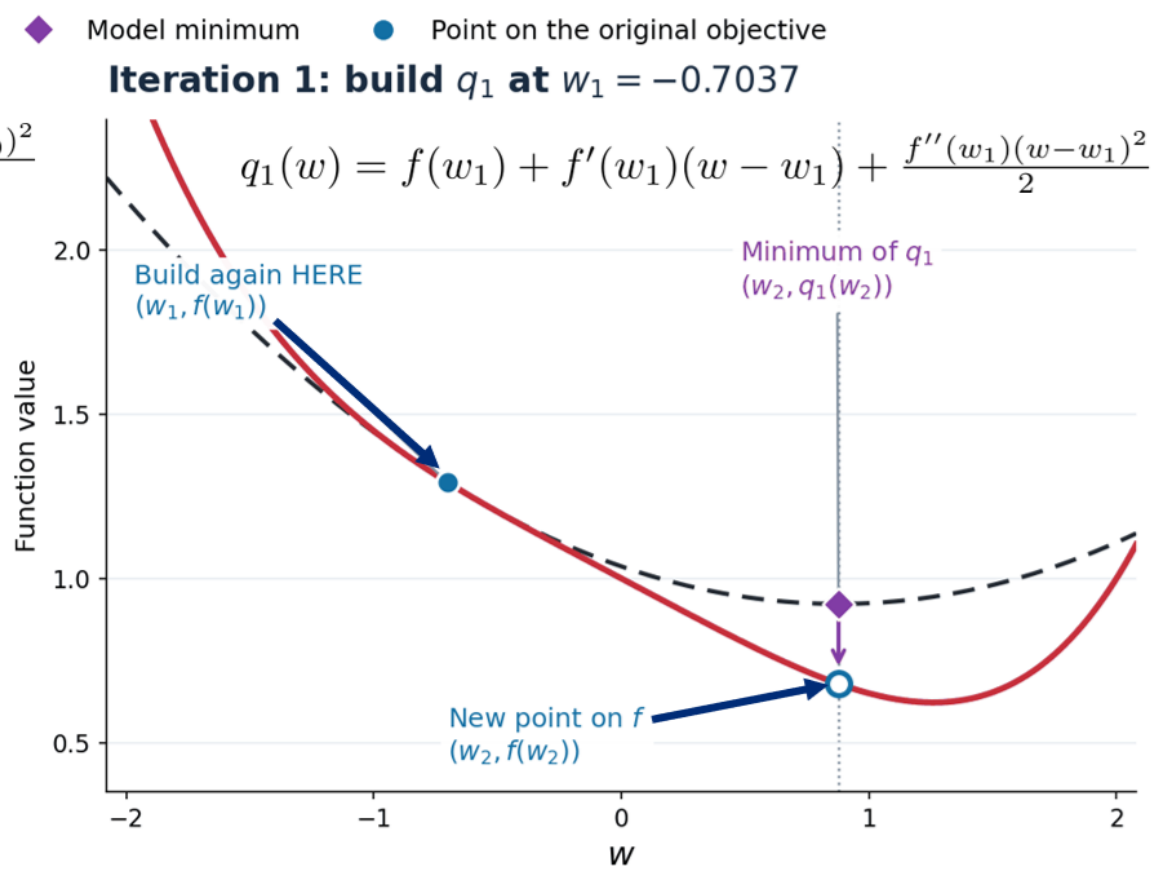


Minimizing the (quadratic) 2nd order Taylor approximation of $f(\mathbf{w})$ at $\boldsymbol{w}=\boldsymbol{w}_k$ will lead to this unique global minimizer, when $\nabla^2 f(\boldsymbol{w}(k)) \succ 0$

An Example of Newton's Method: 2 Iterations

The objective $f(w) = \frac{1}{20}w^4 - \frac{2}{5}w + 1$ stays the same. Each iteration builds a new quadratic model.





$$q_0'(w) = f'(w_0) + f''(w_0)(w - w_0) = 0 \Rightarrow w_1 = -0.7037$$

Evaluate f, f', f'' at w_1

Use these NEW values to build q_1.

$$q_k'(w) = 0 \Rightarrow \; w_{k+1} \triangleq w_k - \frac{f'(w_k)}{f''(w_k)}$$

$$q_1'(w) = f'(w_1) + f''(w_1)(w - w_1) = 0 \Rightarrow w_2 = 0.8771$$

Evaluate f, f', f'' at w_2

Use these NEW values to build q_2, and repeat.

The vertex selects the next horizontal coordinate. The next model is built from the original function at that coordinate.

Full Newton steps ($\alpha = 1$). Here $f''(w_0)>0$ and $f''(w_1)>0$. Subscript k labels the iteration.

Recap: Steepest Descent vs. Newton's Method---in Algebraic Form

Newton's Method

$$\boldsymbol{w}(k+1) = \boldsymbol{w}(k) - \left[\nabla^2 f(\boldsymbol{w}(k))\right]^{-1} \nabla f(\boldsymbol{w}(k))$$

$$\begin{bmatrix} \frac{\partial^2 f}{\partial w_1^2} & \frac{\partial^2 f}{\partial w_1 \partial w_2} & \cdots & \frac{\partial^2 f}{\partial w_1 \partial w_n} \\ \frac{\partial^2 f}{\partial w_2 \partial w_1} & \frac{\partial^2 f}{\partial w_2^2} & \cdots & \frac{\partial^2 f}{\partial w_2 \partial w_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial w_n \partial w_1} & \frac{\partial^2 f}{\partial w_n \partial w_2} & \cdots & \frac{\partial^2 f}{\partial w_n^2} \end{bmatrix}^{-1}$$

Fast convergence (quadratic) when it is near a local minimum point

How do I know if I am near a local minimum point (the Newton's direction is not always descent)

Relatively costly to compute

Steepest Descent

$$\boldsymbol{w}(k+1) = \boldsymbol{w}(k) - \alpha \cdot I \cdot \nabla f(\boldsymbol{w}(k))$$

$$\begin{bmatrix} 1 & 0 & \mathbf{0} & 0 \\ 0 & 1 & \mathbf{0} & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \mathbf{0} & 1 \end{bmatrix}$$

Slow convergence (linear, at best)

The direction is always descent

Quasi-Newton's Method

Relatively cheap to compute

$$\boldsymbol{w}(k+1) = \boldsymbol{w}(k) - \alpha \cdot H_k \cdot \nabla f(\boldsymbol{w}(k))$$

Hope for: (1) H_k gives me better convergence than I $\quad H_k \approx \left[\nabla^2 f(\boldsymbol{w}(k))\right]^{-1}$

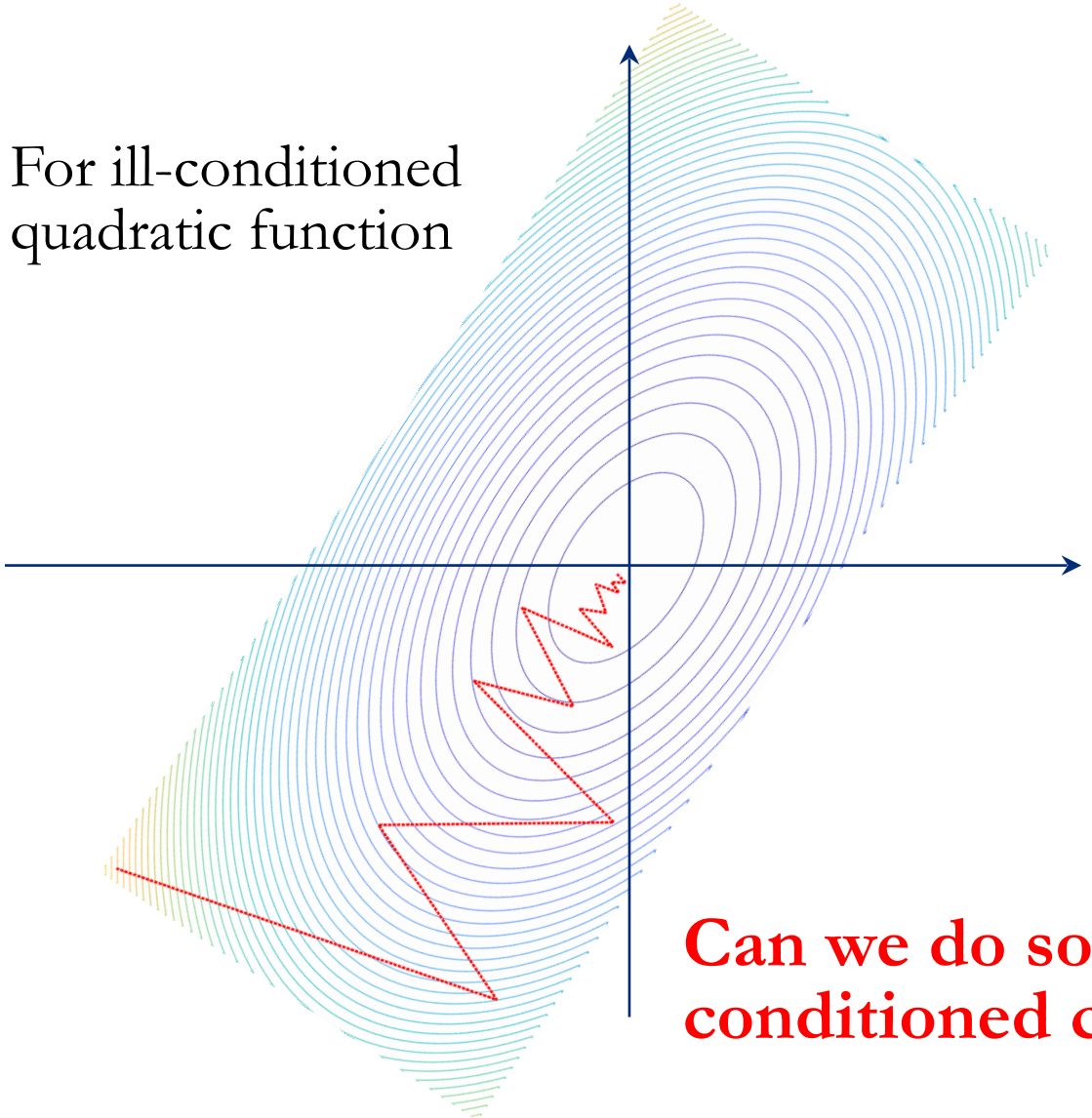
(2) H_k is cheaper to update and apply than an exact Newton step

(3) H_k helps preserve a descent direction

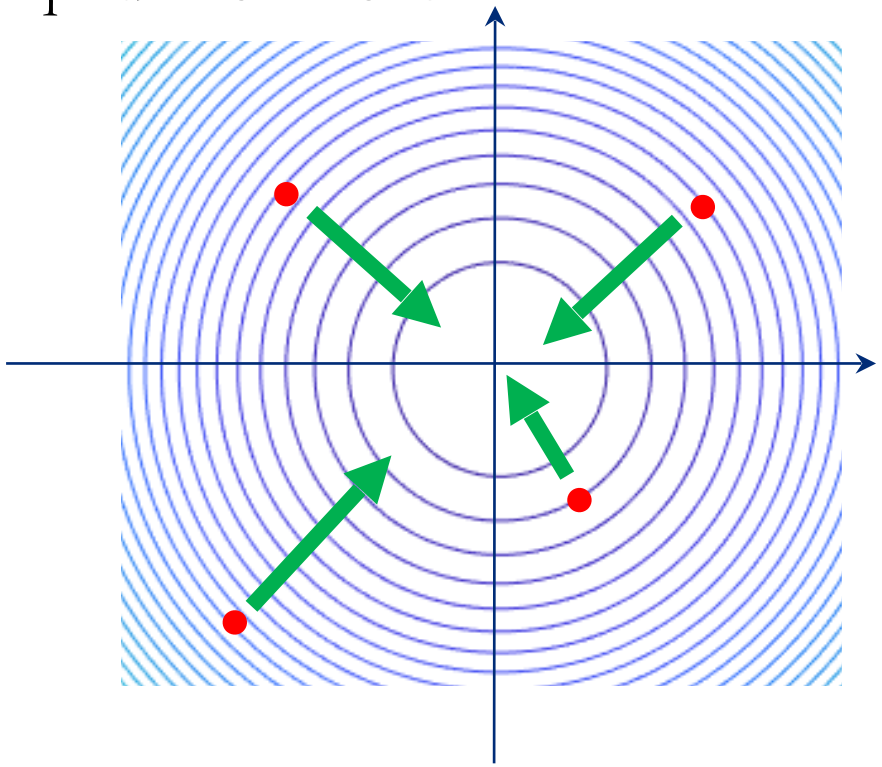
Recap: Steepest Descent vs. Newton's Method---Geometrically

Steepest Descent $\boldsymbol{w}(k+1) = \boldsymbol{w}(k) - \alpha \cdot I \cdot \nabla f(\boldsymbol{w}(k))$

For ill-conditioned quadratic function



For perfectly-conditioned quadratic function



Can we do some clever rescaling to transform ill-conditioned contour curves into a better one?

Steepest Descent After Transformation

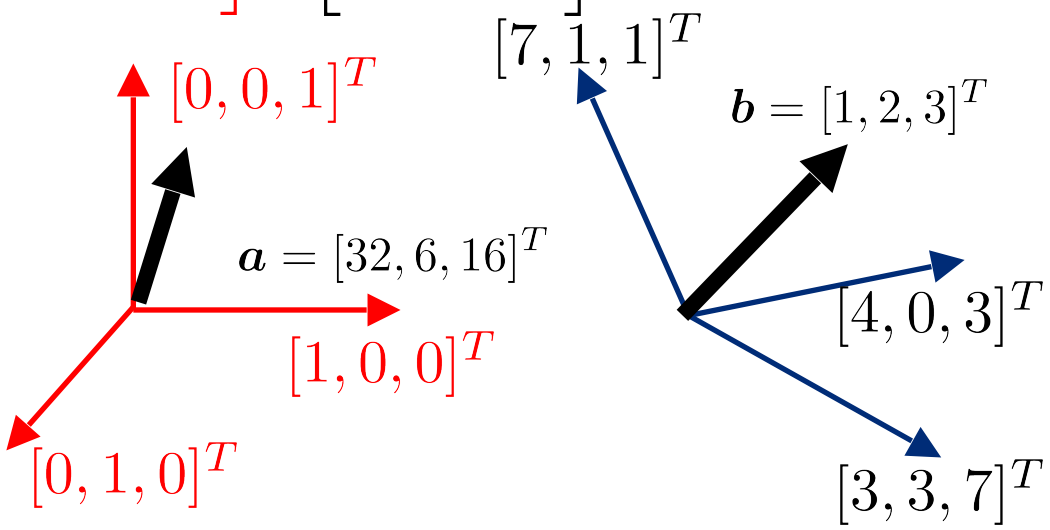
The transformation of the coordinate system is achieved by a matrix

$M \cdot \boldsymbol{b}$

You can view matrix-vector multiplication as: declaring a vector $\boldsymbol{b}$ under the coordinate system specified by the columns of the matrix $\boldsymbol{M}$.

$I \cdot \boldsymbol{a} = M \cdot \boldsymbol{b}$ means: vector $\boldsymbol{b}$ under the coordinate system specified by the columns of matrix $\boldsymbol{M}$ look the same as vector a under the "canonical coordinate system"

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad \begin{bmatrix} 3 & 4 & 7 \\ 3 & 0 & 1 \\ 7 & 3 & 1 \end{bmatrix}$$



$$\min_{\boldsymbol{w}} \ f(\boldsymbol{w})$$

Rescale space: $\boldsymbol{w} = A\boldsymbol{u}$

A is positive definite and invertible

$$\boldsymbol{u} = A^{-1}\boldsymbol{w}$$

Define: $g(\boldsymbol{u}) = f(\boldsymbol{w}) = f(A\boldsymbol{u})$

$$\boldsymbol{u}^* = \arg\min_{\boldsymbol{u}} \ g(\boldsymbol{u})$$

$$\boldsymbol{w}^* = \arg\min_{\boldsymbol{w}} \ f(\boldsymbol{w})$$

$$g(\boldsymbol{u}^*) = f(\boldsymbol{w}^*)$$

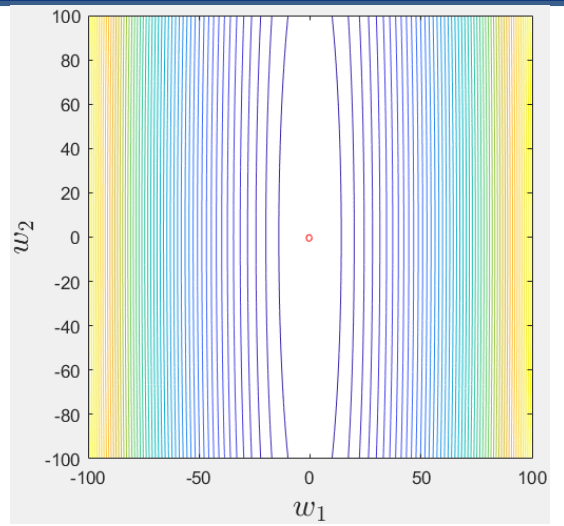
$$\boldsymbol{w}^* = A\boldsymbol{u}^*$$

$$A^{-1}\boldsymbol{w}^* = \boldsymbol{u}^*$$

Will one problem be easier to solve than the other under steepest descent?

Steepest Descent After Transformation: Example

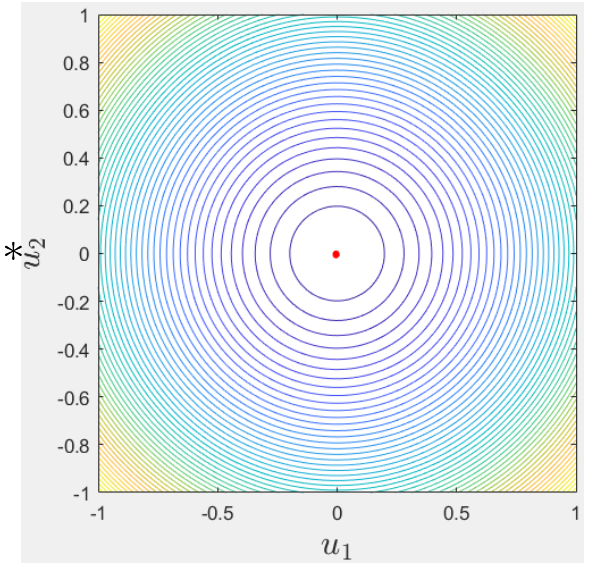
$$f(\boldsymbol{w}) = \frac{1}{2}\boldsymbol{w}^T Q \boldsymbol{w}, \text{ where } Q = \begin{pmatrix} 100 & 0 \\ 0 & 1 \end{pmatrix} \qquad f(\boldsymbol{w}) = \frac{1}{2}(100w_1^2 + w_2^2)$$



Let $A = \begin{pmatrix} 1/10 & 0 \\ 0 & 1 \end{pmatrix}$ $\quad \boldsymbol{w} = A\boldsymbol{u}$ $\quad \boldsymbol{u} = A^{-1}\boldsymbol{w}$

$$g(\boldsymbol{u}) = f(\boldsymbol{w}) = f(A\boldsymbol{u}) = \frac{1}{2}\boldsymbol{u}^T A^T Q A \boldsymbol{u} = \frac{1}{2}\boldsymbol{u}^T \begin{pmatrix} 1/10 & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 100 & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 1/10 & 0 \\ 0 & 1 \end{pmatrix} \boldsymbol{u}$$

$$= \frac{1}{2}\boldsymbol{u}^T \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \boldsymbol{u} = \frac{1}{2}\boldsymbol{u}^T \boldsymbol{u} = (u_1^2 + u_2^2)/2$$



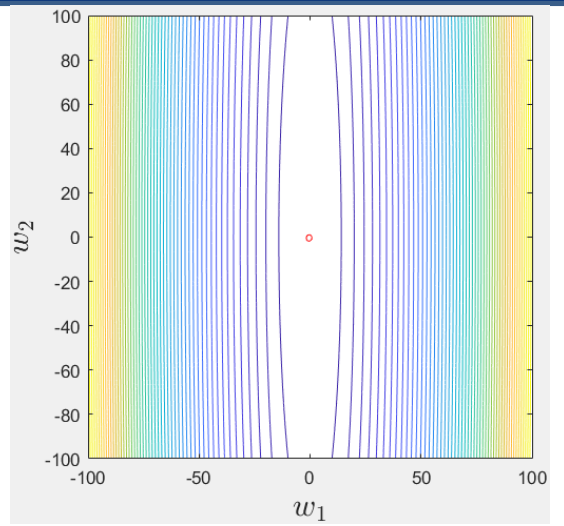
For **any** $\boldsymbol{u}(0)$, $\quad \boldsymbol{u}(1) = \boldsymbol{u}(0) - 1 \cdot \nabla g(\boldsymbol{u}(0)) = \boldsymbol{u}(0) - \boldsymbol{u}(0) = \mathbf{0} = \boldsymbol{u}^*$

Steepest descent needs 1 step to converge $\qquad \boldsymbol{w}^* = A\boldsymbol{u}^* = \mathbf{0}$

Note that in this case, this matrix $A \Rightarrow g(\boldsymbol{u}) = \frac{1}{2}\boldsymbol{u}^T\boldsymbol{u} \Rightarrow \nabla^2 g(\boldsymbol{u}) = I$

Steepest Descent After Transformation: Example

Steepest descent for $f(\boldsymbol{w}) = \frac{1}{2}\boldsymbol{w}^T Q \boldsymbol{w}$, where $Q = \begin{pmatrix} 100 & 0 \\ 0 & 1 \end{pmatrix}$



Say $\boldsymbol{w}(0) = (1/2, 1/2)^T$

$$\boldsymbol{w}(1) = \boldsymbol{w}(0) - \alpha \nabla f(\boldsymbol{w}(0)) = \boldsymbol{w}(0) - \alpha Q \boldsymbol{w}(0)$$

$$= \begin{pmatrix} w_1(0) \\ w_2(0) \end{pmatrix} - \alpha \begin{pmatrix} 100 & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} w_1(0) \\ w_2(0) \end{pmatrix} = \begin{pmatrix} w_1(0) \\ w_2(0) \end{pmatrix} - \alpha \begin{pmatrix} 100 w_1(0) \\ w_2(0) \end{pmatrix} = \begin{pmatrix} (1 - 100\alpha) w_1(0) \\ (1 - \alpha) w_2(0) \end{pmatrix}$$

$$= \begin{pmatrix} (1 - 100\alpha)/2 \\ (1 - \alpha)/2 \end{pmatrix} \neq \begin{pmatrix} 0 \\ 0 \end{pmatrix} \triangleq \boldsymbol{w}^* \text{ for } \textbf{any } \alpha > 0. \text{ So not optimal in 1 iteration}$$

Steepest Descent is NOT Invariant to Linear Transformation

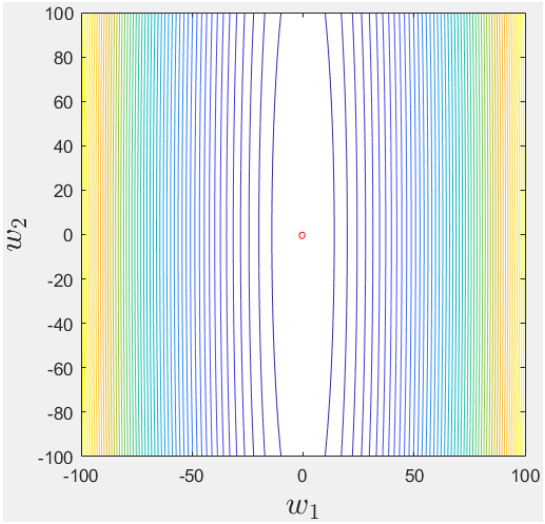
$$f(\boldsymbol{w}) = \frac{1}{2}(100w_1^2 + w_2^2)$$

Let $A = \begin{pmatrix} 1/10 & 0 \\ 0 & 1 \end{pmatrix}$

$$g(\boldsymbol{u}) = \frac{1}{2}(u_1^2 + u_2^2)$$

Hessian of f : $\nabla^2 f(\boldsymbol{w}) = \begin{pmatrix} 100 & 0 \\ 0 & 1 \end{pmatrix}$

Hessian of g : $\nabla^2 g(\boldsymbol{u}) = I$



$$\min_{\boldsymbol{w}} \; f(\boldsymbol{w})$$

$$\boldsymbol{w} = A\boldsymbol{u}$$
$$\boldsymbol{u} = A^{-1}\boldsymbol{w}$$

$$g(\boldsymbol{u}) = f(A\boldsymbol{u})$$

$\boldsymbol{w}(0)$

$\boldsymbol{u}(0) = A^{-1}\boldsymbol{w}(0)$

$\boldsymbol{w}(1)$

$\boldsymbol{u}(1)$

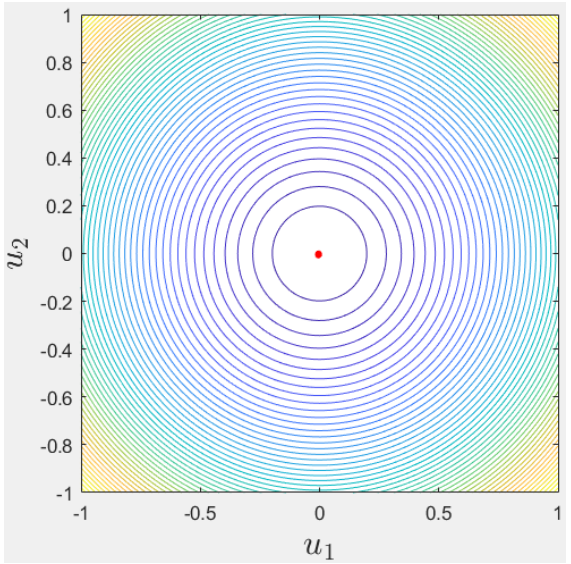
$\boldsymbol{w}(k)$

$\boldsymbol{u}(k)$

$\boldsymbol{w}^*$

$\boldsymbol{u}^* = A^{-1}\boldsymbol{w}^*$

Although the starting and end points are linked by the transformation, the intermediate trajectories are not in general



Linear transformation has the potential to greatly speed up Steepest Descent by making the problem less ill-conditioned

Let's try to find the "right" transformation matrix that makes the problem less ill-conditioned

What is a Good Transformation? (When f is Quadratic and Strictly Convex)

We need matrix A so that $g(\boldsymbol{u}) = f(\boldsymbol{w}),$ where $\boldsymbol{w} = A\boldsymbol{u}$

$\Rightarrow \nabla g(\boldsymbol{u}) = A^T \nabla f(A\boldsymbol{u})$ and $\nabla^2 g(\boldsymbol{u}) = A^T \left[\nabla^2 f(\boldsymbol{w})\right] A$

Our goal is: $\nabla^2 g(\boldsymbol{u}) = I$ So we take $A = \left[\nabla^2 f(\boldsymbol{w})\right]^{-1/2}$

e.g., Hessian of f : $\nabla^2 f(\boldsymbol{w}) = \begin{pmatrix} 100 & 0 \\ 0 & 1 \end{pmatrix}$

$A = \begin{pmatrix} 1/10 & 0 \\ 0 & 1 \end{pmatrix} = \left[\nabla^2 f(\boldsymbol{w})\right]^{-1/2}$

Lemma: If a matrix M is positive definite, then $M^{-1/2}$ exists and is also symmetric with strictly positive eigenvalues.

Observation: Updating the steepest descent in the space of u for $g(u)$

$$\boldsymbol{u}_{k+1} = \boldsymbol{u}_k - \alpha \nabla g(\boldsymbol{u}_k)$$

$$A\boldsymbol{u}_{k+1} = A[\boldsymbol{u}_k - \alpha \nabla g(\boldsymbol{u}_k)]$$

$$\boldsymbol{w}_{k+1} = \boldsymbol{w}_k - \alpha A A^T \nabla f(A\boldsymbol{u}_k), \text{ since } \nabla g(\boldsymbol{u}) = A^T \nabla f(A\boldsymbol{u})$$

$$\boldsymbol{w}_{k+1} = \boldsymbol{w}_k - \alpha \left[\nabla^2 f(\boldsymbol{w}_k)\right]^{-1} \nabla f(\boldsymbol{w}_k), \text{ since } A = \left[\nabla^2 f(\boldsymbol{w})\right]^{-1/2}$$

is updating the Newton's method with a step size α in the space of w for $f(w)$

Newton's Method: From Quadratic to General Functions

Strictly convex quadratic *f*

The Hessian is constant.

$$A = [\nabla^2 f(\boldsymbol{w})]^{-1/2}$$

One fixed transformation for all iterations

General *f*

The Hessian can change with position.

$$A_k = [\nabla^2 f(\boldsymbol{w}_k)]^{-1/2}$$

Recompute the transformation at each iteration

At iteration k: choose A_k, then hold it fixed for this step.

$$\boldsymbol{w} = A_k \boldsymbol{u} \qquad g_k(\boldsymbol{u}) = f(A_k \boldsymbol{u}) \qquad \boldsymbol{u}_k = A_k^{-1} \boldsymbol{w}_k$$

$$\nabla^2 g_k(\boldsymbol{u}_k) = A_k^{\mathrm{T}} [\nabla^2 f(\boldsymbol{w}_k)] A_k = I$$ at the current point

One steepest descent step, mapped back to w:

$$A_k[\boldsymbol{u}_k - \alpha \nabla g_k(\boldsymbol{u}_k)] = \boldsymbol{w}_k - \alpha A_k A_k^{\mathrm{T}} \nabla f(\boldsymbol{w}_k) = \boldsymbol{w}_k - \alpha [\nabla^2 f(\boldsymbol{w}_k)]^{-1} \nabla f(\boldsymbol{w}_k)$$

At $\boldsymbol{w}_{k+1}$: recompute A_{k+1}, re-express f, and repeat.

The objective *f* stays the same. The coordinate system is updated each step.

Assume $\nabla^2 f(\boldsymbol{w}_k) > 0$ at the points considered.

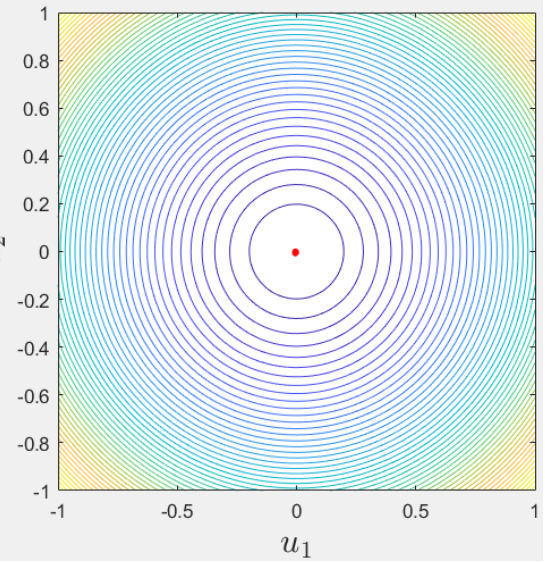
Quasi-Newton's Method

Newton's Method

$$\boldsymbol{w}(k+1) = \boldsymbol{w}(k) - \left[\nabla^2 f(\boldsymbol{w}(k))\right]^{-1} \nabla f(\boldsymbol{w}(k))$$

Steepest Descent

$$\boldsymbol{w}(k+1) = \boldsymbol{w}(k) - \alpha \cdot I \cdot \nabla f(\boldsymbol{w}(k))$$

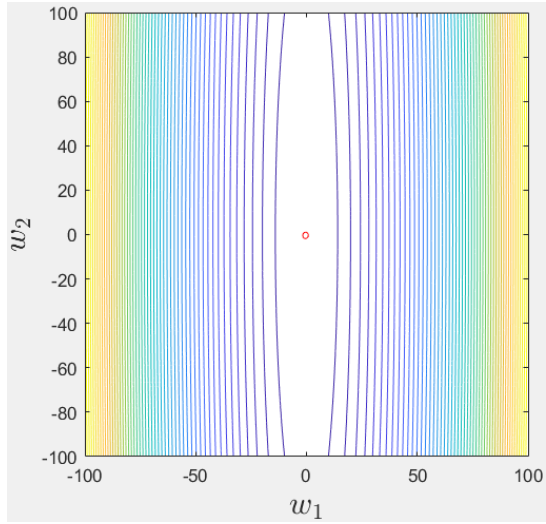


Knows everything about the curvature

Completely agnostic about the curvature

Quasi-Newton's Method

$$\boldsymbol{w}(k+1) = \boldsymbol{w}(k) - \alpha \cdot H_k \cdot \nabla f(\boldsymbol{w}(k))$$



Hope for: (1) H_k gives me better convergence than I

(2) H_k has lower computational cost than an exact Newton

(3) H_k helps preserve a descent direction

Rescale the contour curves to something that is "close" to perfect circles but balance the computation cost

We are going to build a ***proxy*** of the Newton's full Hessian matrix inverse recursively, using the curvature information along the generated path. (curvature measures the change of the gradient)

Quasi-Newton's Method

$$f(\boldsymbol{w}+\boldsymbol{p}) \approx m(\boldsymbol{p}) \triangleq f(\boldsymbol{w}) + \nabla f(\boldsymbol{w})^T \boldsymbol{p} + \frac{1}{2}\boldsymbol{p}^T B \boldsymbol{p} \qquad \text{If } B = \nabla^2 f(\boldsymbol{w}), \text{ then it's Newton}$$

$$\boldsymbol{p} = -B^{-1}\nabla f(\boldsymbol{w}), \text{ where } B \succ 0$$

Wolfe Condition

All Quasi-Newton methods: $\boldsymbol{w}(k+1) = \boldsymbol{w}(k) - \alpha_k \cdot B_k^{-1}\nabla f(\boldsymbol{w}(k))$

Update B_k iteratively (allows a rich B_k without re-compute it from scratch in every iteration)

Properties Required: $B_k^{-1}\nabla f(\boldsymbol{w}(k))$ is easy to compute (easier than Newton)

B_k captures the curvature information of f along the path.

$$\nabla f(\boldsymbol{w}+\boldsymbol{\alpha p}) = \nabla f(\boldsymbol{w}) + \nabla^2 f(\boldsymbol{w})\alpha\boldsymbol{p} + o\|\alpha\boldsymbol{p}\|$$

High order small term, can be ignored

Let $\boldsymbol{w}(k+1) = \boldsymbol{w}(k) + \alpha_k \boldsymbol{p}$

$$\nabla f(\boldsymbol{w}(k+1)) = \nabla f(\boldsymbol{w}(k)) + \nabla^2 f(\boldsymbol{w}(k))[\boldsymbol{w}(k+1) - \boldsymbol{w}(k)] + o\|\boldsymbol{w}(k+1) - \boldsymbol{w}(k)\|$$

$$\nabla^2 f(\boldsymbol{w}(k))[\boldsymbol{w}(k+1) - \boldsymbol{w}(k)] \approx \nabla f(\boldsymbol{w}(k+1)) - \nabla f(\boldsymbol{w}(k))$$

We construct B_{k+1} so that $B_{k+1}[\boldsymbol{w}(k+1) - \boldsymbol{w}(k)] = \nabla f(\boldsymbol{w}(k+1)) - \nabla f(\boldsymbol{w}(k))$

Broyden, Fletcher, Goldfarb, and Shanno (BFGS) Algorithm

Properties Required:

1. B_k is symmetric (Because Hessian is a symmetric matrix and we hope B_k captures the curvature information of f)

2. Secant Equation: $B_{k+1}\underbrace{[\boldsymbol{w}(k+1)-\boldsymbol{w}(k)]}_{\boldsymbol{s}_k} = \underbrace{\nabla f(\boldsymbol{w}(k+1))-\nabla f(\boldsymbol{w}(k))}_{\boldsymbol{y}_k} \Rightarrow B_{k+1}\boldsymbol{s}_k = \boldsymbol{y}_k$

With the secant equation alone is not able to pin down B_{k+1}, since it has $n(n+1)/2$ number of entries (symmetry) that need to be specified, and the secant equation only provides n equations

3. We require B_{k+1} to be "close" to B_k (Make the smallest change that incorporates the new curvature information)

$$B_{k+1} \triangleq \arg\min_{B} \|B - B_k\|$$

$$\text{s.t.} \quad B = B^T, \text{ symmetry}$$

$$B\boldsymbol{s}_k = \boldsymbol{y}_k, \text{ secant condition}$$

Different norm leads to different algorithms (different B_{k+1})

Broyden, Fletcher, Goldfarb, and Shanno (BFGS) Algorithm

$$B_{k+1} \triangleq \arg\min_{B} \|B - B_k\|$$

$$\text{s.t.} \quad B = B^T, \text{ symmetry}$$

$$B\boldsymbol{s}_k = \boldsymbol{y}_k, \text{ secant condition}$$

With " $B_k^{-1}\nabla f(\boldsymbol{w}(k))$ should be easy to compute" in mind

$$\boldsymbol{w}(k+1) = \boldsymbol{w}(k) - \alpha_k \cdot B_k^{-1}\nabla f(\boldsymbol{w}(k))$$

$$H_{k+1} \triangleq \arg\min_{H} \|H - H_k\|$$

$$\text{s.t.} \quad H = H^T$$

$$\boldsymbol{s}_k = H\boldsymbol{y}_k$$

$BFGS: \ H_k = B_k^{-1}$

1. H_{k+1} is close to H_k
2. H_k is symmetric
3. H_{k+1} satisfies secant equation $\boldsymbol{s}_k = H_{k+1}\boldsymbol{y}_k$

BFGS uses weighted Frobenius Norm $\|A\|_F^2 = \sum_{i,j} A_{ij}^2 \ (Frobenius\ Norm)$

$$\|A\|_Z^2 = \|Z^{1/2}AZ^{1/2}\|_F^2 \ (Weighted\ Frobenius\ Norm)$$

BFGS uses a special of the weight matrix Z such that H_{k+1} can be derived very easily from H_k

$$H_{k+1} = (I - \rho_k \boldsymbol{s}_k \boldsymbol{y}_k^T) H_k (I - \rho_k \boldsymbol{y}_k \boldsymbol{s}_k^T) + \rho_k \boldsymbol{s}_k \boldsymbol{s}_k^T, \text{ where } \rho_k = \frac{1}{\boldsymbol{y}_k^T \boldsymbol{s}_k}$$

$$\boldsymbol{s}_k \triangleq \boldsymbol{w}_{k+1} - \boldsymbol{w}_k$$

$$\boldsymbol{y}_k \triangleq \nabla f(\boldsymbol{w}(k+1)) - \nabla f(\boldsymbol{w}(k))$$

Broyden, Fletcher, Goldfarb, and Shanno (BFGS) Algorithm

Step 1. Initialize $\boldsymbol{w}(0)$ and $H_0 = I$

Step 2. For iteration $k = 0,1,2,\ldots$

Repeat until some stopping criterion is met;

(a) Pick a descent direction to go $\boldsymbol{p}(k) = -H_k \nabla f(\boldsymbol{w}(k))$

(b) Pick the step size a_k using Wolfe condition

$$\boldsymbol{w}(k+1) = \boldsymbol{w}(k) - \alpha_k \cdot H_k \nabla f(\boldsymbol{w}(k))$$

(c) Update $\boldsymbol{s}_k \triangleq \boldsymbol{w}(k+1) - \boldsymbol{w}(k)$

$\boldsymbol{y}_k \triangleq \nabla f(\boldsymbol{w}(k+1)) - \nabla f(\boldsymbol{w}(k))$

$H_k \succ 0$ gives a descent direction; *Wolfe* conditions preserve positive definiteness through the BFGS update

BFGS avoids computing Hessian and its inverse in each iteration

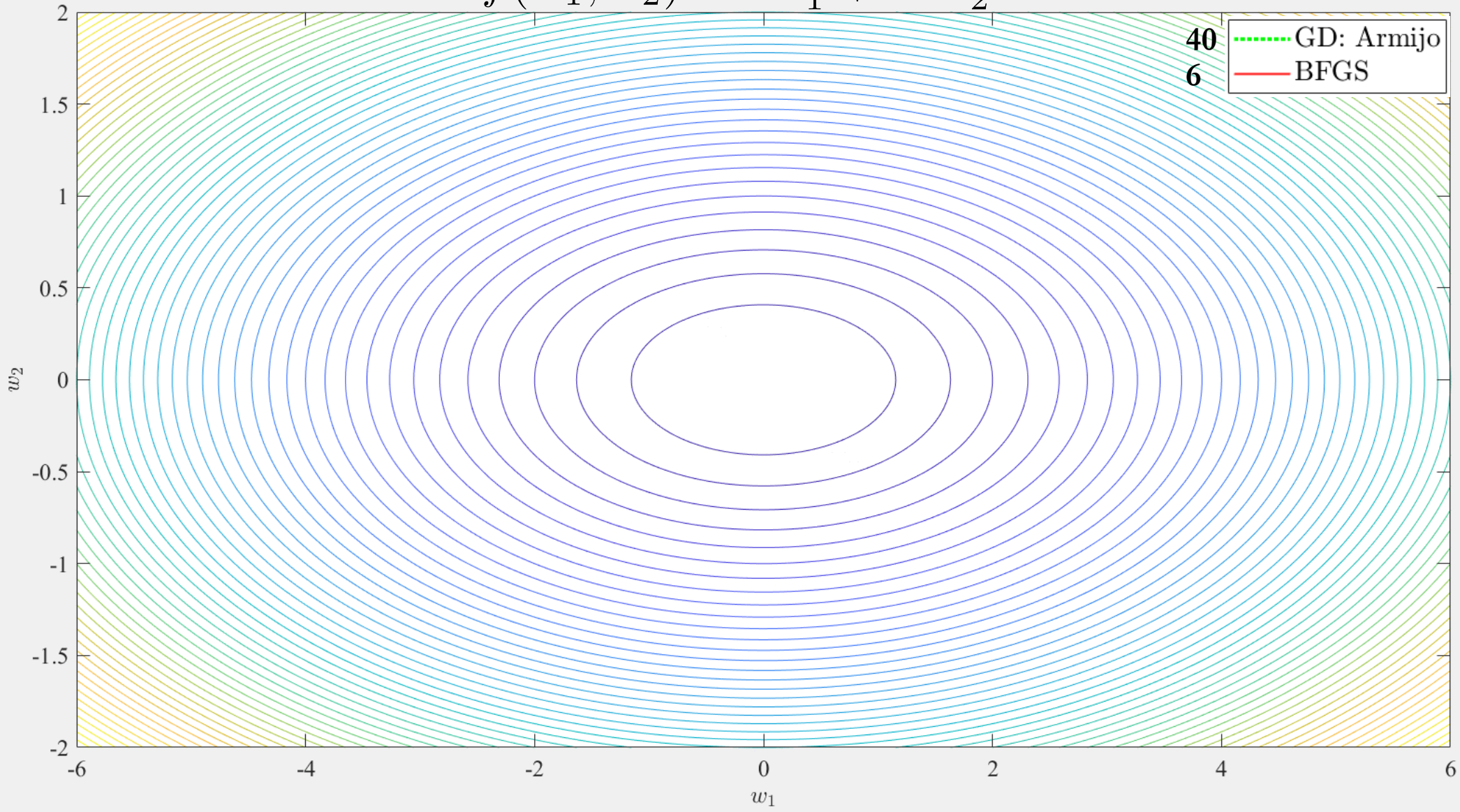
If $\boldsymbol{y}_k^T \boldsymbol{s}_k > \epsilon_{skip} \|\boldsymbol{y}_k\| \|\boldsymbol{s}_k\|$, where ϵ_{skip} is some small number, e.g., 10^{-8}

$$H_{k+1} = (I - \rho_k \boldsymbol{s}_k \boldsymbol{y}_k^T) H_k (I - \rho_k \boldsymbol{y}_k \boldsymbol{s}_k^T) + \rho_k \boldsymbol{s}_k \boldsymbol{s}_k^T, \text{ where } \rho_k = \frac{1}{\boldsymbol{y}_k^T \boldsymbol{s}_k}$$

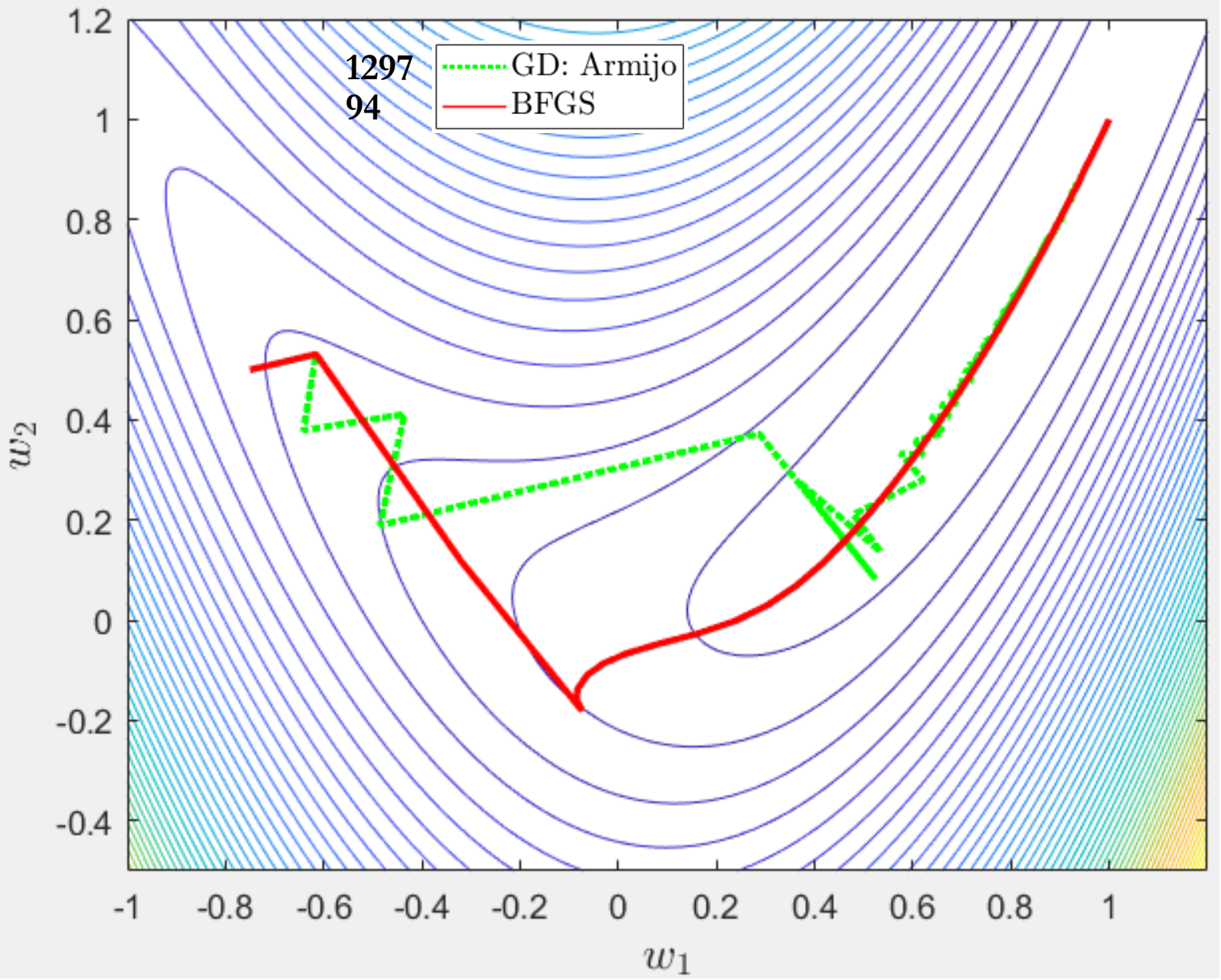
Otherwise, skip the matrix update: $H_{k+1} = H_k$

Broyden, Fletcher, Goldfarb, and Shanno (BFGS) Algorithm

$$f(w_1, w_2) = 3w_1^2 + 24w_2^2$$



Broyden, Fletcher, Goldfarb, and Shanno (BFGS) Algorithm



Rosenbrock Function

$$f(\boldsymbol{w}) = 10(w_2 - w_1^2)^2 + (1 - w_1)^2$$

With initial point (-3/4, 1/2)

Limited Memory BFGS (L-BFGS)

BFGS matrix update and direction computation: $O(n^2)$

Nevertheless, H_k is a dense matrix (n by n)

Observation: to compute H_{k+1}, we only need $\boldsymbol{s}_k$, $\boldsymbol{y}_k$, given H_k

$$H_{k+1} = (I - \rho_k \boldsymbol{s}_k \boldsymbol{y}_k^T) H_k (I - \rho_k \boldsymbol{y}_k \boldsymbol{s}_k^T) + \rho_k \boldsymbol{s}_k \boldsymbol{s}_k^T, \text{ where } \rho_k = \frac{1}{\boldsymbol{y}_k^T \boldsymbol{s}_k}$$

Let $\boldsymbol{g} = \nabla f(\boldsymbol{w}(k+1))$

$$H_{k+1}\boldsymbol{g} = (I - \rho_k \boldsymbol{s}_k \boldsymbol{y}_k^T) H_k (\boldsymbol{g} - \boldsymbol{y}_k \rho_k \boldsymbol{s}_k^T \boldsymbol{g}) + \rho_k \boldsymbol{s}_k \boldsymbol{s}_k^T \boldsymbol{g}$$

a

$\boldsymbol{q}$

$\beta = \rho_k \boldsymbol{y}_k^T \boldsymbol{r}$

$r = H_k q$

$$\Leftrightarrow \quad H_{k+1}\boldsymbol{g} = \boldsymbol{r} + \boldsymbol{s}_k(a - \beta)$$

Limited Memory BFGS (L-BFGS): Implicit Form BFGS

$$H_{k+1}\boldsymbol{g} = (I - \rho_k \boldsymbol{s}_k \boldsymbol{y}_k^T) H_k (\boldsymbol{g} - \boldsymbol{y}_k \rho_k \boldsymbol{s}_k^T \boldsymbol{g}) + \rho_k \boldsymbol{s}_k \boldsymbol{s}_k^T \boldsymbol{g} \qquad \text{Let } \boldsymbol{g} = \nabla f(\boldsymbol{w}(k+1))$$

Instead of storing H_{k+1} (n by n matrix), one can store all $\boldsymbol{s}_t, \boldsymbol{y}_t$ for $t=0,1,2,\ldots,k$, given $H_0=I$

Recursion (from $t=k$ to $t=1$); gives 2-loop computation of $H_{k+1}\nabla f(\boldsymbol{w}(k+1))$

$$H_{k+1}\boldsymbol{g} = \boldsymbol{r} + \boldsymbol{s}_k(a - \beta)$$

1. Let $\boldsymbol{q} = \nabla f(\boldsymbol{w}(k+1))$
2. For $t = k, k\text{-}1, k\text{-}2,\ldots,1,0$
   (a) Compute $a_t = \dfrac{\boldsymbol{s}_t^T \boldsymbol{q}}{\boldsymbol{y}_t^T \boldsymbol{s}_t}$
   (b) Update $\boldsymbol{q} = \boldsymbol{q} - a_t \boldsymbol{y}_t$
3. Let $\boldsymbol{r} = H_0 \boldsymbol{q}$
4. For $t = 0,1,2,\ldots,k\text{-}1,k$
   (a) Let $\beta_t = \dfrac{\boldsymbol{y}_t^T \boldsymbol{r}}{\boldsymbol{y}_t^T \boldsymbol{s}_t}$
   (b) Update $\boldsymbol{r} = \boldsymbol{r} + \boldsymbol{s}_t(a_t - \beta_t)$

Storage for vector pairs: $2n(k+1)$, growing with the number of updates

An equivalent full-history implementation whose storage and computation grow with the history length. Keeping only recent pairs leads to L-BFGS.

Limited Memory BFGS (L-BFGS): Implementation

Recall: Full-BFGS update: $H_{k+1}\boldsymbol{g} = (I - \rho_k \boldsymbol{s}_k \boldsymbol{y}_k^T) H_k (\boldsymbol{g} - \boldsymbol{y}_k \rho_k \boldsymbol{s}_k^T \boldsymbol{g}) + \rho_k \boldsymbol{s}_k \boldsymbol{s}_k^T \boldsymbol{g}$ Let $\boldsymbol{g} = \nabla f(\boldsymbol{w}(k+1))$

Recursion (from t=k to t=l, where $l = \max\{0,$ ***k-m+1,*** $\}$and m is buffer size);
gives 2-loop computation of $H_{k+1}\boldsymbol{g}$

BFGS but look only ***m*** iterations back in history

1. Let $\boldsymbol{q} = \nabla f(\boldsymbol{w}(k+1))$ and $l = \max\{0, k-m+1\}$
2. For $t = k, k\text{-}1,\ldots,l$
   (a) Compute $a_t = \dfrac{\boldsymbol{s}_t^T \boldsymbol{q}}{\boldsymbol{y}_t^T \boldsymbol{s}_t}$
   (b) Update $\boldsymbol{q} = \boldsymbol{q} - a_t \boldsymbol{y}_t$
3. Let $\boldsymbol{r} = \dfrac{\boldsymbol{s}_k^T \boldsymbol{y}_k}{\boldsymbol{y}_k^T \boldsymbol{y}_k} \boldsymbol{q}$
4. For $t = l,\ldots,k\text{-}1,k$
   (a) Let $\beta_t = \dfrac{\boldsymbol{y}_t^T \boldsymbol{r}}{\boldsymbol{y}_t^T \boldsymbol{s}_t}$
   (b) Update $\boldsymbol{r} = \boldsymbol{r} + \boldsymbol{s}_t (a_t - \beta_t)$

Pair storage: at most **2nm** *numbers. Direction computation: O(nm)*
(e.g., *m* =*20*)

Recent pairs provide curvature information along the latest part of the path. L-BFGS retains a fixed number of these pairs to construct a useful approximation while controlling storage and computational cost.

Summary

Iterative Methods for Unconstrained Optimization $\min_{\boldsymbol{w}\in\mathbb{R}^n} \; f(\boldsymbol{w})$

Step 1. Initialize $\boldsymbol{w}(0)$

Step 2. For iteration $k = 0,1,2,\ldots$ Repeat until some stopping criterion is met;

(a) Choose a search direction ***p(k)***

1st-order method: Steepest Descent (with Momentum), SGD: slow convergence; low storage; inexpensive direction updates

$$\boldsymbol{p}(k) = -H_k \nabla f(\boldsymbol{w}(k))$$

2st-order method: Newton, Quasi-Newton (BFGS, L-BFGS), fast convergence; low storage; inexpensive direction updates

(b) Pick a step size α For descent directions: ***Wolfe Condition***

Armijo: $f(\boldsymbol{w}(k) + \alpha^{(t)}\boldsymbol{p}(k)) \leq f(\boldsymbol{w}(k)) + \alpha^{(t)} c_1 \cdot \boldsymbol{p}^T(k)\nabla f(\boldsymbol{w}_k)$

Curvature: $\nabla f(\boldsymbol{w}(k) + \alpha^{(t)}\boldsymbol{p}(k))^T \boldsymbol{p}(k) \geq c_2 \cdot \boldsymbol{p}^T(k)\nabla f(\boldsymbol{w}_k)$

where $0 < c_1 < c_2 < 1$ $\quad e.g.,\ c_1 = 10^{-4},\ c_2 = 0.9$

(c) Update $\boldsymbol{w}(k+1) = \boldsymbol{w}(k) + \alpha_k \boldsymbol{p}(k)$

Global Convergence towards stationarity, under regularity conditions

Summary

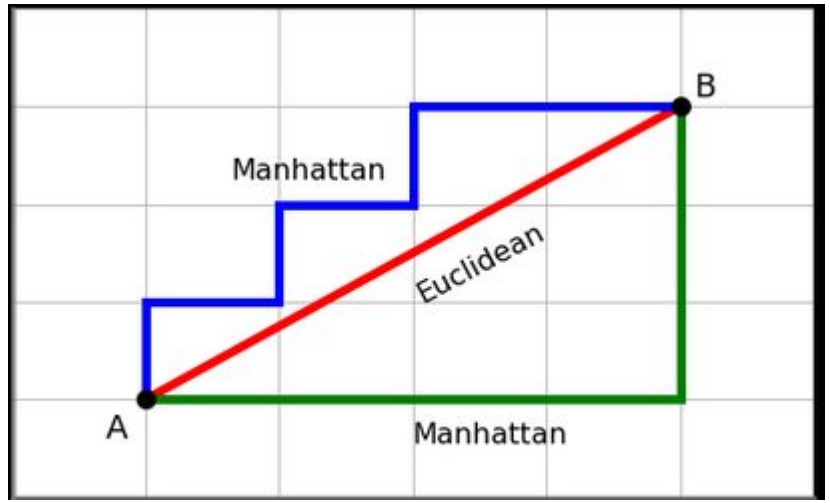
Many optimization methods can be viewed as steepest descent under a suitably chosen norm

The negative gradient points to the direction that the objective function decreases in the steepest way under the Euclidean norm.

$$\boldsymbol{p}(k) = -H_k \nabla f(\boldsymbol{w}(k)), \;\; H_k \succ 0$$

$$\|\boldsymbol{p}\|_{H_k^{-1}} = \sqrt{\boldsymbol{p}^T H_k^{-1} \boldsymbol{p}}$$

$\boldsymbol{p}_k$ is a steepest-descent direction under this norm.
The norm may change from one iteration to the next.



Why not just stay with the most common norm, the Euclidean norm?